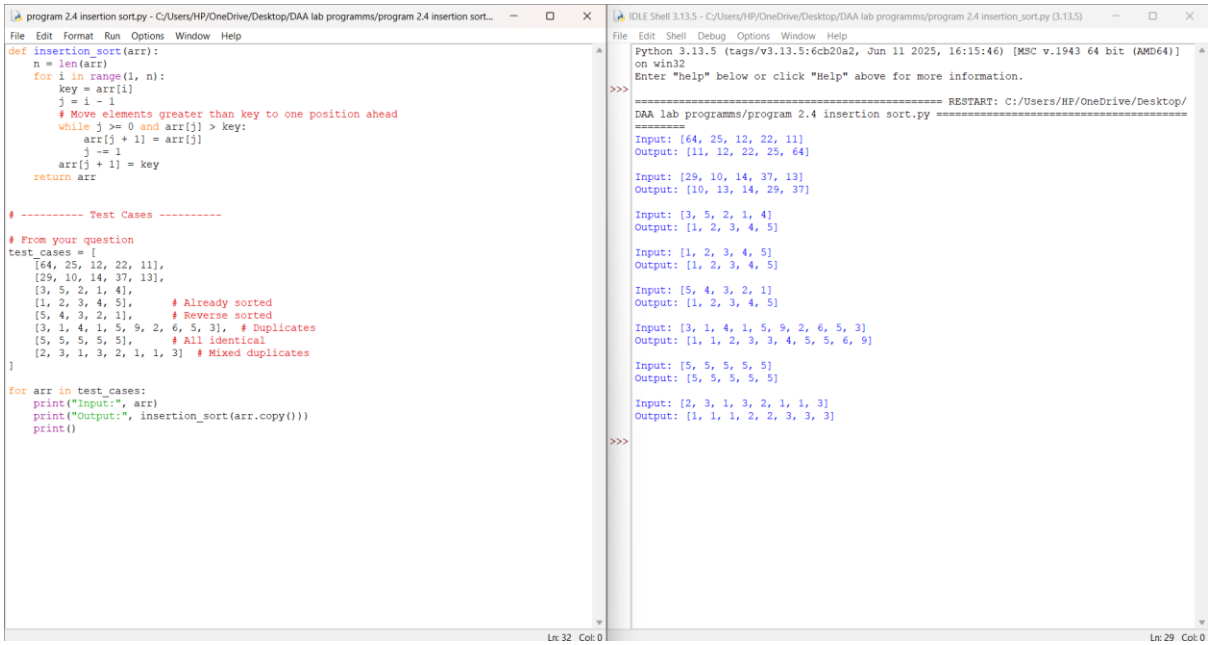
DESIGN AND ANALYSIS OF ALGORITHM

1.



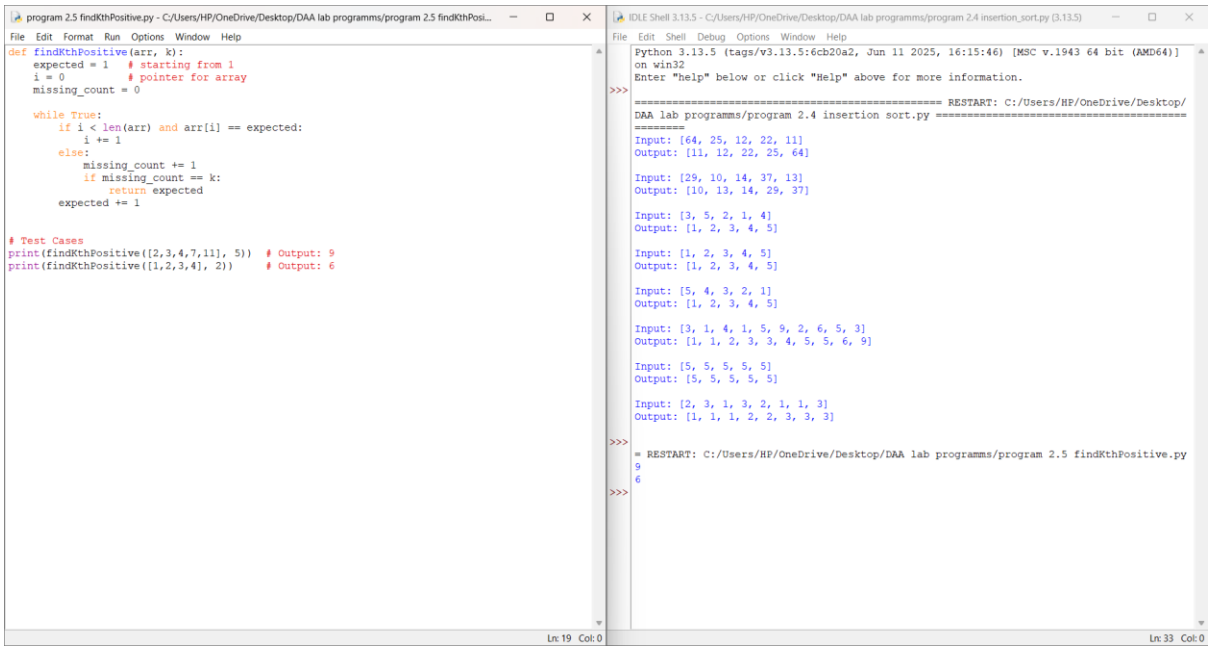
```
program 2.4 insertion sort.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.4 insertion sort...
File  Edit  Format  Run  Options  Window  Help
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        # Move elements greater than key to one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr


# ---------- Test Cases ----------

# From your question
test_cases = [
    [64, 25, 12, 22, 11],
    [29, 10, 14, 37, 13],
    [3, 5, 2, 1, 4],
    [1, 2, 3, 4, 5],      # Already sorted
    [5, 4, 3, 2, 1],      # Reverse sorted
    [3, 1, 4, 1, 5, 9, 2, 6, 5, 3],  # Duplicates
    [5, 5, 5, 5, 5],      # All identical
    [2, 3, 1, 3, 2, 1, 1, 3]  # Mixed duplicates
]

for arr in test_cases:
    print("Input:", arr)
    print("Output:", insertion_sort(arr.copy()))
    print()
```

Ln: 32 Col: 0

```
IDLE Shell 3.13.5 - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.4 insertion_sort.py (3.13.5)
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/HP/OneDrive/Desktop/
DAA lab programms/program 2.4 insertion sort.py ==========
========
Input: [64, 25, 12, 22, 11]
Output: [11, 12, 22, 25, 64]

Input: [29, 10, 14, 37, 13]
Output: [10, 13, 14, 29, 37]

Input: [3, 5, 2, 1, 4]
Output: [1, 2, 3, 4, 5]

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

Input: [5, 4, 3, 2, 1]
Output: [1, 2, 3, 4, 5]

Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Input: [5, 5, 5, 5, 5]
Output: [5, 5, 5, 5, 5]

Input: [2, 3, 1, 3, 2, 1, 1, 3]
Output: [1, 1, 1, 2, 2, 3, 3, 3]

>>>
```

Ln: 29 Col: 0

2.



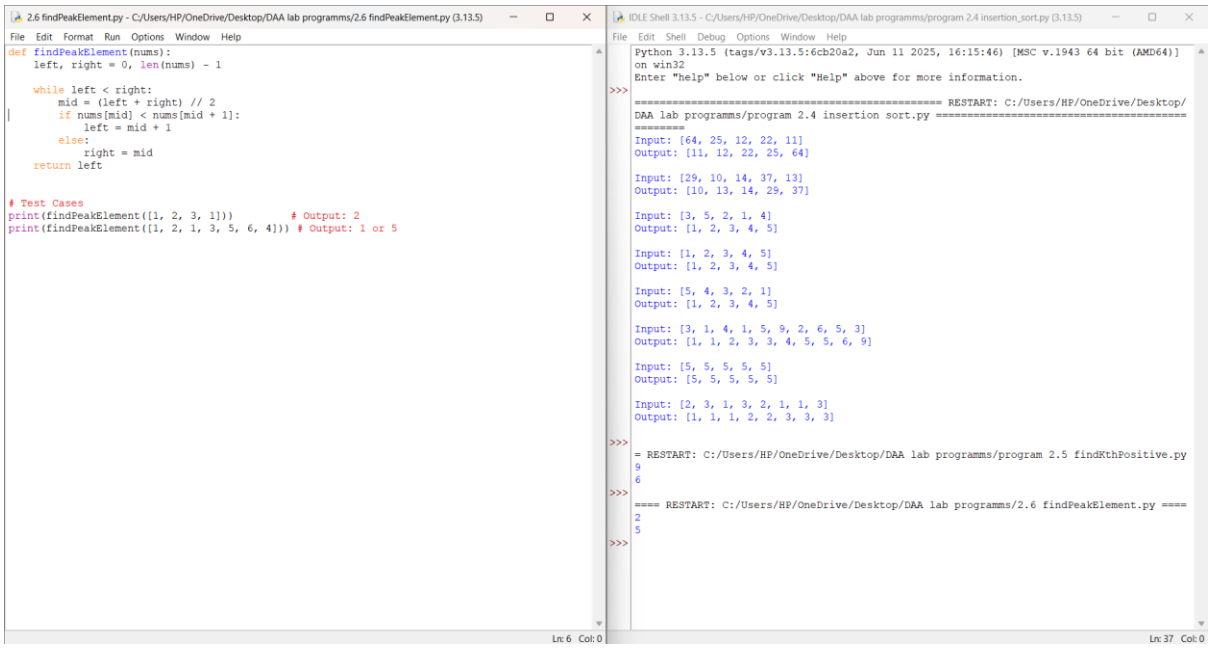
```
program 2.5 findKthPositive.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.5 findKthPosi...
File  Edit  Format  Run  Options  Window  Help
def findKthPositive(arr, k):
    expected = 1   # starting from 1
    i = 0          # pointer for array
    missing_count = 0

    while True:
        if i < len(arr) and arr[i] == expected:
            i += 1
        else:
            missing_count += 1
            if missing_count == k:
                return expected
        expected += 1


# Test Cases
print(findKthPositive([2,3,4,7,11], 5))  # Output: 9
print(findKthPositive([1,2,3,4], 2))     # Output: 6
```

Ln: 19 Col: 0

```
IDLE Shell 3.13.5 - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.4 insertion_sort.py (3.13.5)
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/HP/OneDrive/Desktop/
DAA lab programms/program 2.4 insertion sort.py ==========
========
Input: [64, 25, 12, 22, 11]
Output: [11, 12, 22, 25, 64]

Input: [29, 10, 14, 37, 13]
Output: [10, 13, 14, 29, 37]

Input: [3, 5, 2, 1, 4]
Output: [1, 2, 3, 4, 5]

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

Input: [5, 4, 3, 2, 1]
Output: [1, 2, 3, 4, 5]

Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Input: [5, 5, 5, 5, 5]
Output: [5, 5, 5, 5, 5]

Input: [2, 3, 1, 3, 2, 1, 1, 3]
Output: [1, 1, 1, 2, 2, 3, 3, 3]

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.5 findKthPositive.py
9
6
>>>
```

Ln: 33 Col: 0

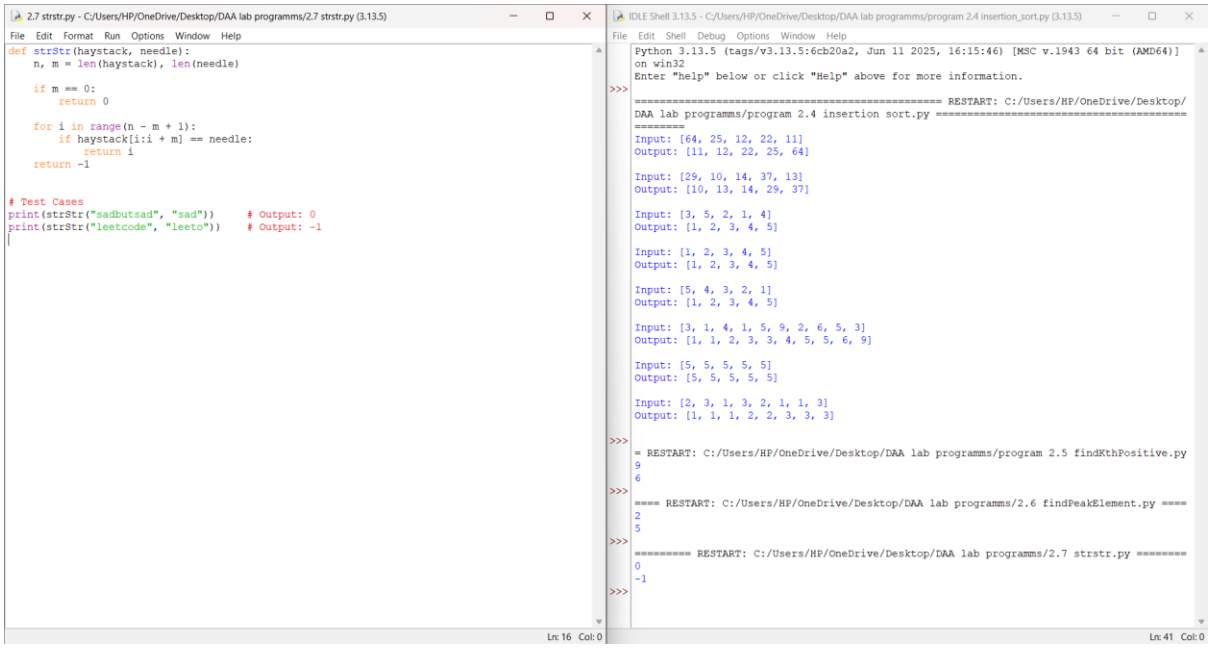
3.



```
2.6 findPeakElement.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/2.6 findPeakElement.py (3.13.5)
File  Edit  Format  Run  Options  Window  Help
def findPeakElement(nums):
    left, right = 0, len(nums) - 1

    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left


# Test Cases
print(findPeakElement([1, 2, 3, 1]))          # Output: 2
print(findPeakElement([1, 2, 1, 3, 5, 6, 4])) # Output: 1 or 5
```

```
IDLE Shell 3.13.5 - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.4 insertion_sort.py (3.13.5)
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/HP/OneDrive/Desktop/
DAA lab programms/program 2.4 insertion sort.py ==========
========
Input: [64, 25, 12, 22, 11]
Output: [11, 12, 22, 25, 64]

Input: [29, 10, 14, 37, 13]
Output: [10, 13, 14, 29, 37]

Input: [3, 5, 2, 1, 4]
Output: [1, 2, 3, 4, 5]

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

Input: [5, 4, 3, 2, 1]
Output: [1, 2, 3, 4, 5]

Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Input: [5, 5, 5, 5, 5]
Output: [5, 5, 5, 5, 5]

Input: [2, 3, 1, 3, 2, 1, 1, 3]
Output: [1, 1, 1, 2, 2, 3, 3, 3]

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.5 findKthPositive.py
9
6
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/2.6 findPeakElement.py ====
2
5
>>>
```

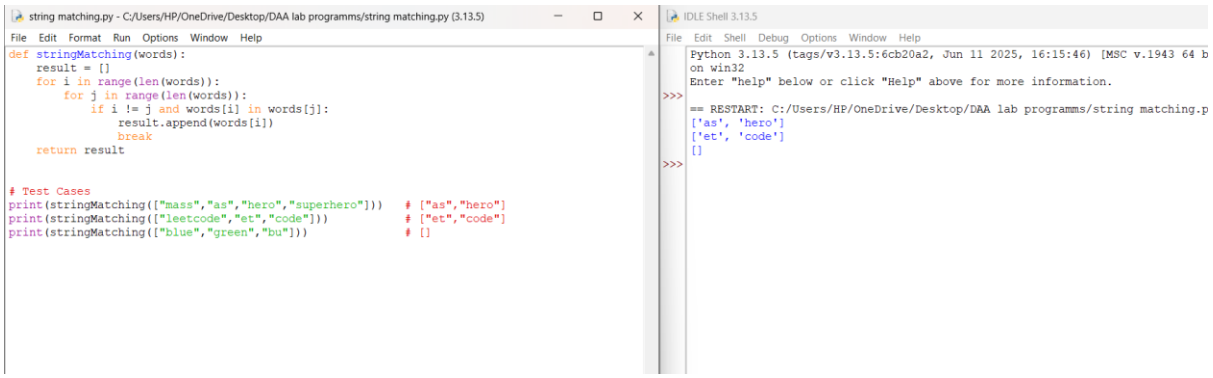
4.



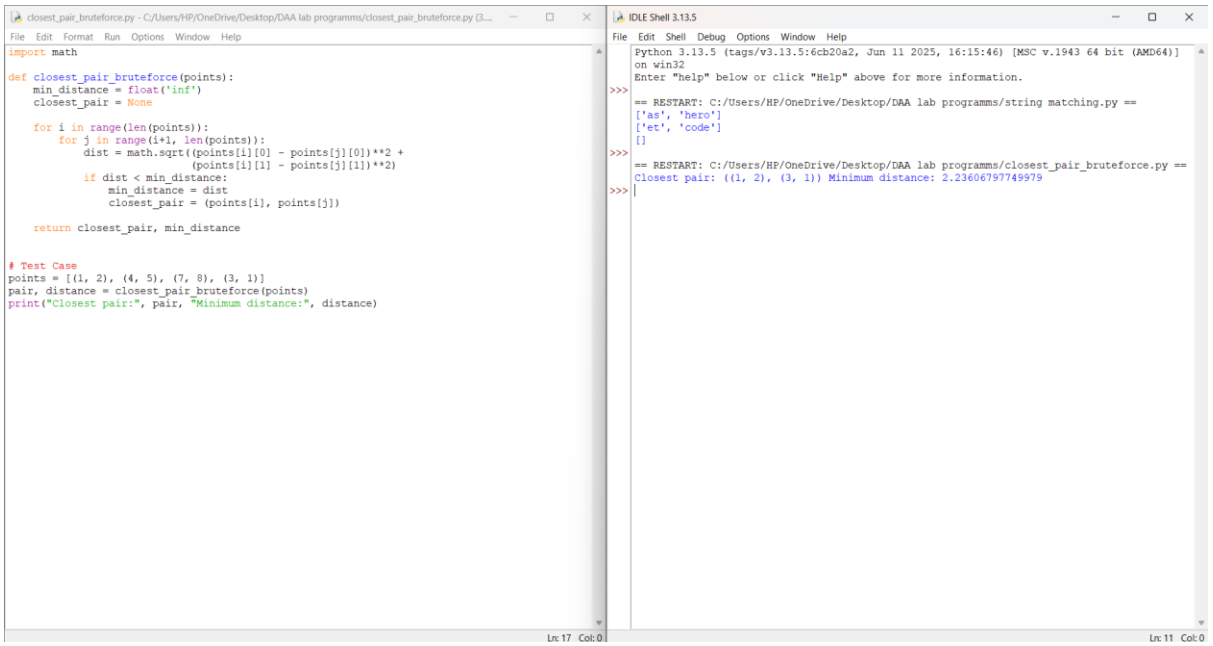
```
2.7 strstr.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/2.7 strstr.py (3.13.5)
File  Edit  Format  Run  Options  Window  Help
def strStr(haystack, needle):
    n, m = len(haystack), len(needle)

    if m == 0:
        return 0

    for i in range(n - m + 1):
        if haystack[i:i + m] == needle:
            return i
    return -1


# Test Cases
print(strStr("sadbutsad", "sad"))     # Output: 0
print(strStr("leetcode", "leeto"))    # Output: -1
```

```
IDLE Shell 3.13.5 - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.4 insertion_sort.py (3.13.5)
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/HP/OneDrive/Desktop/
DAA lab programms/program 2.4 insertion sort.py ==========
========
Input: [64, 25, 12, 22, 11]
Output: [11, 12, 22, 25, 64]

Input: [29, 10, 14, 37, 13]
Output: [10, 13, 14, 29, 37]

Input: [3, 5, 2, 1, 4]
Output: [1, 2, 3, 4, 5]

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

Input: [5, 4, 3, 2, 1]
Output: [1, 2, 3, 4, 5]

Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Input: [5, 5, 5, 5, 5]
Output: [5, 5, 5, 5, 5]

Input: [2, 3, 1, 3, 2, 1, 1, 3]
Output: [1, 1, 1, 2, 2, 3, 3, 3]

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.5 findKthPositive.py
9
6
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/2.6 findPeakElement.py ====
2
5
>>>
========== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/2.7 strstr.py ========
0
-1
>>>
```

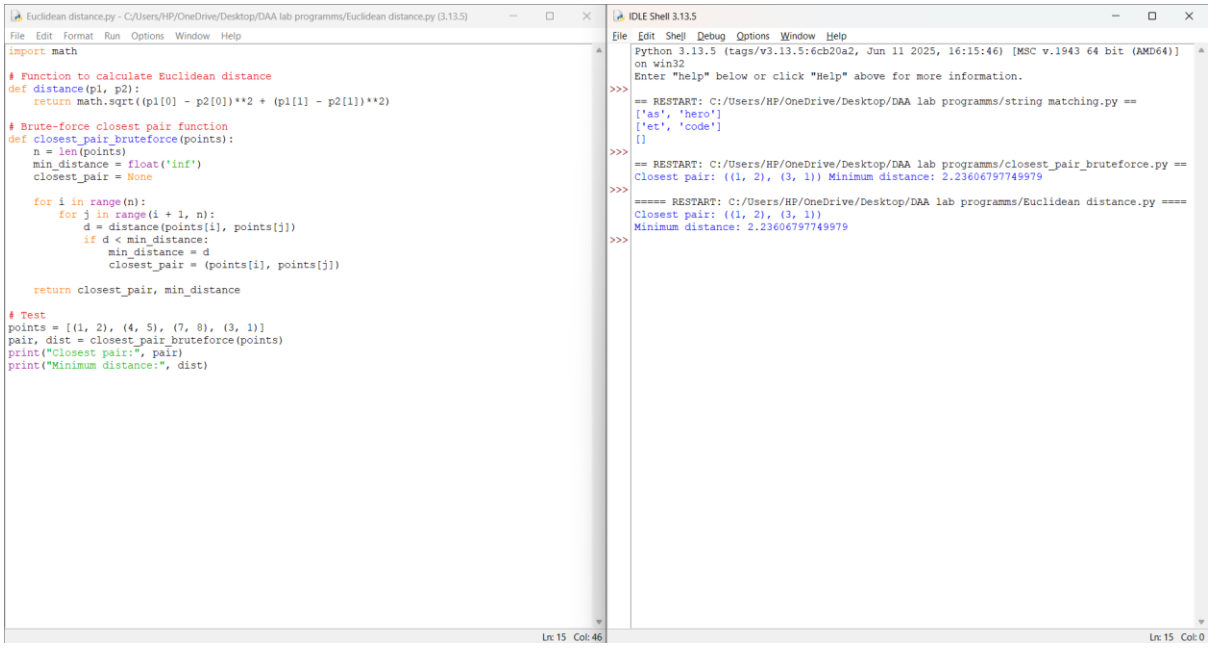
5.


string matching.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/string matching.py (3.13.5)
File Edit Format Run Options Window Help
def stringMatching(words):
result = []
for i in range(len(words)):
for j in range(len(words)):
if i != j and words[i] in words[j]:
result.append(words[i])
break
return result
Test Cases
print(stringMatching(["mass","as","hero","superhero"])) # ["as","hero"]
print(stringMatching(["leetcode","et","code"])) # ["et","code"]
print(stringMatching(["blue","green","bu"])) # []
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 b
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/string matching.p
['as', 'hero']
['et', 'code']
[]
>>>

6.


closest_pair_bruteforce.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/closest_pair_bruteforce.py (3....
File Edit Format Run Options Window Help
import math
def closest_pair_bruteforce(points):
min_distance = float('inf')
closest_pair = None
for i in range(len(points)):
for j in range(i+1, len(points)):
dist = math.sqrt((points[i][0] - points[j][0])**2 +
(points[i][1] - points[j][1])**2)
if dist < min_distance:
min_distance = dist
closest_pair = (points[i], points[j])
return closest_pair, min_distance
# Test Case
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, distance = closest_pair_bruteforce(points)
print("Closest pair:", pair, "Minimum distance:", distance)
Ln: 17 Col: 0
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/string matching.py ==
['as', 'hero']
['et', 'code']
[]
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/closest_pair_bruteforce.py ==
Closest pair: ((1, 2), (3, 1)) Minimum distance: 2.23606797749979
>>>
Ln: 11 Col: 0

7.



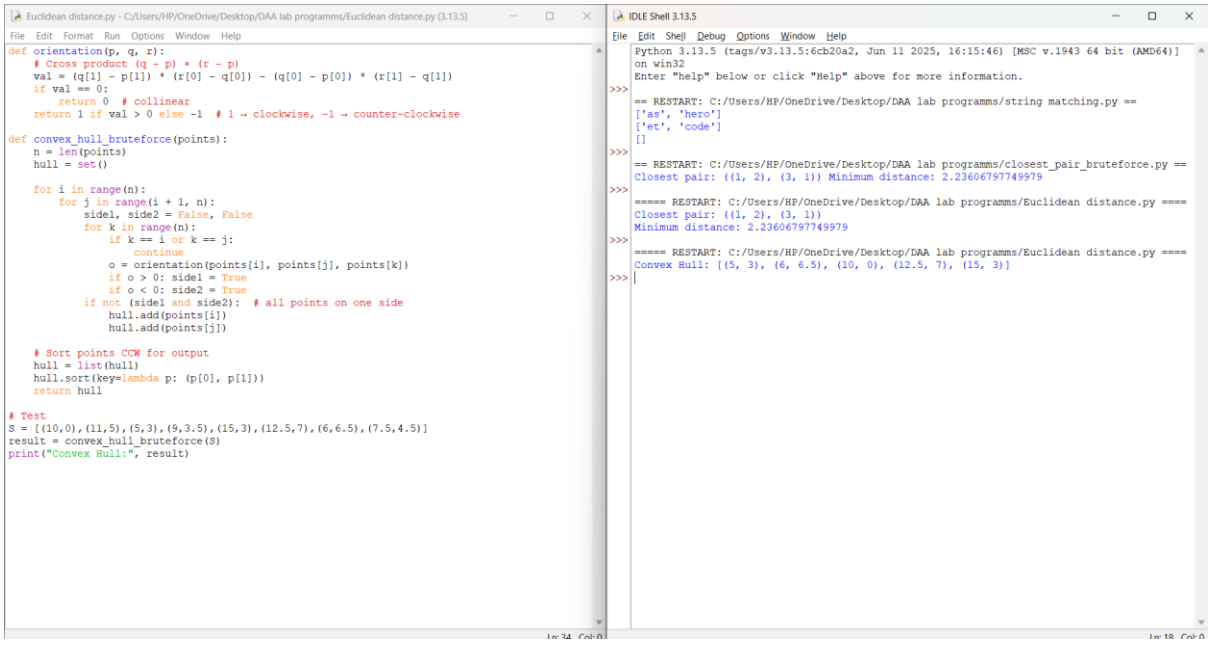
```
Euclidean distance.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py (3.13.5)
File  Edit  Format  Run  Options  Window  Help
import math

# Function to calculate Euclidean distance
def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

# Brute-force closest pair function
def closest_pair_bruteforce(points):
    n = len(points)
    min_distance = float('inf')
    closest_pair = None

    for i in range(n):
        for j in range(i + 1, n):
            d = distance(points[i], points[j])
            if d < min_distance:
                min_distance = d
                closest_pair = (points[i], points[j])

    return closest_pair, min_distance

# Test
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, dist = closest_pair_bruteforce(points)
print("Closest pair:", pair)
print("Minimum distance:", dist)
```

Ln: 15 Col: 46

```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/string matching.py ==
['as', 'hero']
['et', 'code']
[]
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/closest_pair_bruteforce.py ==
Closest pair: ((1, 2), (3, 1)) Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Closest pair: ((1, 2), (3, 1))
Minimum distance: 2.23606797749979
>>>
```

Ln: 15 Col: 0

8.



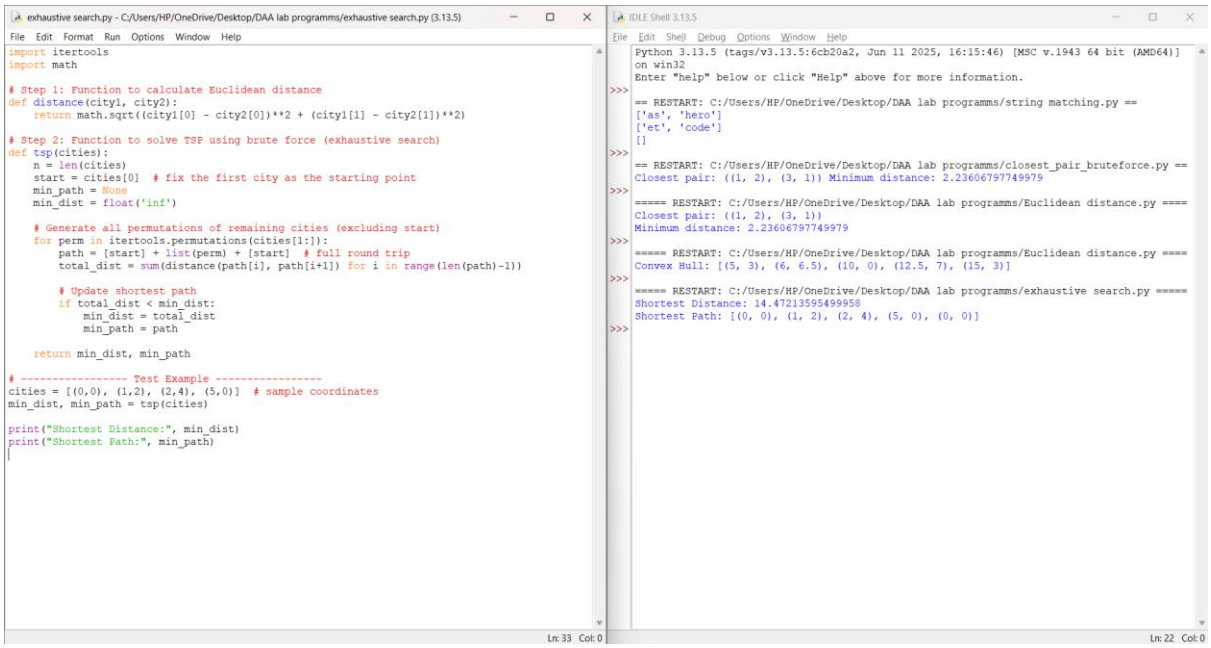
```
Euclidean distance.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py (3.13.5)
File  Edit  Format  Run  Options  Window  Help
def orientation(p, q, r):
    # Cross product (q - p) × (r - p)
    val = (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])
    if val == 0:
        return 0  # collinear
    return 1 if val > 0 else -1  # 1 → clockwise, -1 → counter-clockwise

def convex_hull_bruteforce(points):
    n = len(points)
    hull = set()

    for i in range(n):
        for j in range(i + 1, n):
            side1, side2 = False, False
            for k in range(n):
                if k == i or k == j:
                    continue
                o = orientation(points[i], points[j], points[k])
                if o > 0: side1 = True
                if o < 0: side2 = True
            if not (side1 and side2):  # all points on one side
                hull.add(points[i])
                hull.add(points[j])

    # Sort points CCW for output
    hull = list(hull)
    hull.sort(key=lambda p: (p[0], p[1]))
    return hull

# Test
S = [(10,0),(11,5),(5,3),(9,3.5),(15,3),(12.5,7),(6,6.5),(7.5,4.5)]
result = convex_hull_bruteforce(S)
print("Convex Hull:", result)
```

Ln: 34 Col: 0

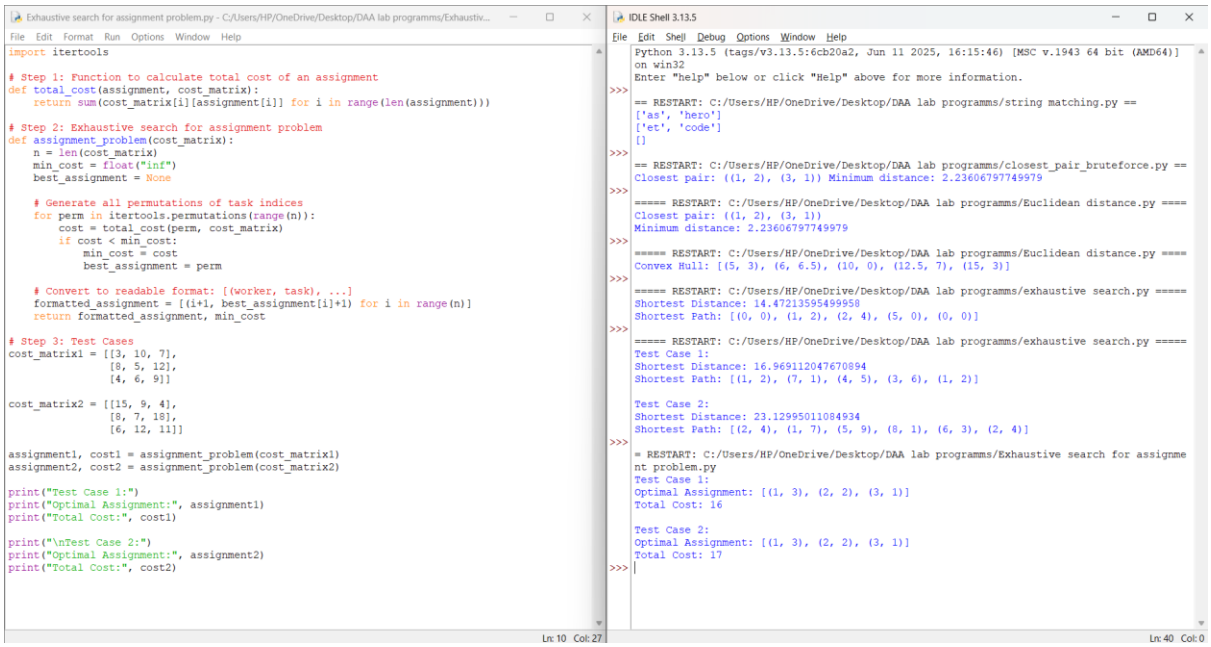
```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/string matching.py ==
['as', 'hero']
['et', 'code']
[]
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/closest_pair_bruteforce.py ==
Closest pair: ((1, 2), (3, 1)) Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Closest pair: ((1, 2), (3, 1))
Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Convex Hull: [(5, 3), (6, 6.5), (10, 0), (12.5, 7), (15, 3)]
>>>
```

Ln: 18 Col: 0

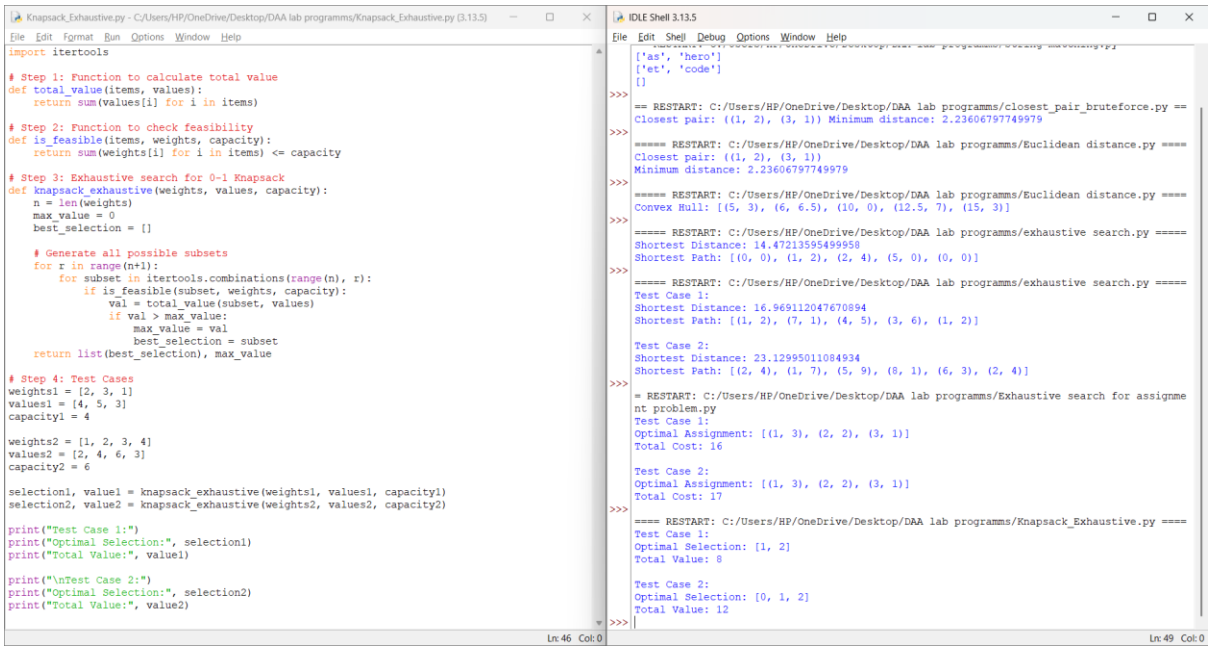
9.


exhaustive search.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/exhaustive search.py (3.13.5)
File Edit Format Run Options Window Help
import itertools
import math
# Step 1: Function to calculate Euclidean distance
def distance(city1, city2):
return math.sqrt((city1[0] - city2[0])**2 + (city1[1] - city2[1])**2)
# Step 2: Function to solve TSP using brute force (exhaustive search)
def tsp(cities):
n = len(cities)
start = cities[0] # fix the first city as the starting point
min_path = None
min_dist = float('inf')
# Generate all permutations of remaining cities (excluding start)
for perm in itertools.permutations(cities[1:]):
path = [start] + list(perm) + [start] # full round trip
total_dist = sum(distance(path[i], path[i+1]) for i in range(len(path)-1))
# Update shortest path
if total_dist < min_dist:
min_dist = total_dist
min_path = path
return min_dist, min_path
# ---------------- Test Example ----------------
cities = [(0,0), (1,2), (2,4), (5,0)] # sample coordinates
min_dist, min_path = tsp(cities)
print("Shortest Distance:", min_dist)
print("Shortest Path:", min_path)
Ln: 33 Col: 0
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/string matching.py ==
['as', 'hero']
['et', 'code']
[]
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/closest_pair_bruteforce.py ==
Closest pair: ((1, 2), (3, 1)) Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Closest pair: ((1, 2), (3, 1))
Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Convex Hull: [(5, 3), (6, 6.5), (10, 0), (12.5, 7), (15, 3)]
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/exhaustive search.py =====
Shortest Distance: 14.47213595499958
Shortest Path: [(0, 0), (1, 2), (2, 4), (5, 0), (0, 0)]
>>>
Ln: 22 Col: 0

10.


Exhaustive search for assignment problem.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/Exhaustiv...
File Edit Format Run Options Window Help
import itertools
# Step 1: Function to calculate total cost of an assignment
def total_cost(assignment, cost_matrix):
return sum(cost_matrix[i][assignment[i]] for i in range(len(assignment)))
# Step 2: Exhaustive search for assignment problem
def assignment_problem(cost_matrix):
n = len(cost_matrix)
min_cost = float("inf")
best_assignment = None
# Generate all permutations of task indices
for perm in itertools.permutations(range(n)):
cost = total_cost(perm, cost_matrix)
if cost < min_cost:
min_cost = cost
best_assignment = perm
# Convert to readable format: [(worker, task), ...]
formatted_assignment = [(i+1, best_assignment[i]+1) for i in range(n)]
return formatted_assignment, min_cost
# Step 3: Test Cases
cost_matrix1 = [[3, 10, 7],
[8, 5, 12],
[4, 6, 9]]
cost_matrix2 = [[15, 9, 4],
[8, 7, 18],
[6, 12, 11]]
assignment1, cost1 = assignment_problem(cost_matrix1)
assignment2, cost2 = assignment_problem(cost_matrix2)
print("Test Case 1:")
print("Optimal Assignment:", assignment1)
print("Total Cost:", cost1)
print("\nTest Case 2:")
print("Optimal Assignment:", assignment2)
print("Total Cost:", cost2)
Ln: 10 Col: 27
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/string matching.py ==
['as', 'hero']
['et', 'code']
[]
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/closest_pair_bruteforce.py ==
Closest pair: ((1, 2), (3, 1)) Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Closest pair: ((1, 2), (3, 1))
Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Convex Hull: [(5, 3), (6, 6.5), (10, 0), (12.5, 7), (15, 3)]
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/exhaustive search.py =====
Shortest Distance: 14.47213595499958
Shortest Path: [(0, 0), (1, 2), (2, 4), (5, 0), (0, 0)]
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/exhaustive search.py =====
Test Case 1:
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]
Test Case 2:
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (1, 7), (5, 9), (8, 1), (6, 3), (2, 4)]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Exhaustive search for assignme
nt problem.py
Test Case 1:
Optimal Assignment: [(1, 3), (2, 2), (3, 1)]
Total Cost: 16
Test Case 2:
Optimal Assignment: [(1, 3), (2, 2), (3, 1)]
Total Cost: 17
>>>
Ln: 40 Col: 0

11.



```
Knapsack_Exhaustive.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/Knapsack_Exhaustive.py (3.13.5)
File  Edit  Format  Run  Options  Window  Help
import itertools

# Step 1: Function to calculate total value
def total_value(items, values):
    return sum(values[i] for i in items)

# Step 2: Function to check feasibility
def is_feasible(items, weights, capacity):
    return sum(weights[i] for i in items) <= capacity

# Step 3: Exhaustive search for 0-1 Knapsack
def knapsack_exhaustive(weights, values, capacity):
    n = len(weights)
    max_value = 0
    best_selection = []

    # Generate all possible subsets
    for r in range(n+1):
        for subset in itertools.combinations(range(n), r):
            if is_feasible(subset, weights, capacity):
                val = total_value(subset, values)
                if val > max_value:
                    max_value = val
                    best_selection = subset
    return list(best_selection), max_value

# Step 4: Test Cases
weights1 = [2, 3, 1]
values1 = [4, 5, 3]
capacity1 = 4

weights2 = [1, 2, 3, 4]
values2 = [2, 4, 6, 3]
capacity2 = 6

selection1, value1 = knapsack_exhaustive(weights1, values1, capacity1)
selection2, value2 = knapsack_exhaustive(weights2, values2, capacity2)

print("Test Case 1:")
print("Optimal Selection:", selection1)
print("Total Value:", value1)

print("\nTest Case 2:")
print("Optimal Selection:", selection2)
print("Total Value:", value2)
```

Ln: 46 Col: 0

```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
['as', 'hero']
['et', 'code']
[]
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/closest_pair_bruteforce.py ==
Closest pair: ((1, 2), (3, 1)) Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Closest pair: ((1, 2), (3, 1))
Minimum distance: 2.23606797749979
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Euclidean distance.py ====
Convex Hull: [(5, 3), (6, 6.5), (10, 0), (12.5, 7), (15, 3)]
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/exhaustive search.py =====
Shortest Distance: 14.47213595499958
Shortest Path: [(0, 0), (1, 2), (2, 4), (5, 0), (0, 0)]
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/exhaustive search.py =====
Test Case 1:
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Test Case 2:
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (1, 7), (5, 9), (8, 1), (6, 3), (2, 4)]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Exhaustive search for assignme
nt problem.py
Test Case 1:
Optimal Assignment: [(1, 3), (2, 2), (3, 1)]
Total Cost: 16

Test Case 2:
Optimal Assignment: [(1, 3), (2, 2), (3, 1)]
Total Cost: 17
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/Knapsack_Exhaustive.py ====
Test Case 1:
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2:
Optimal Selection: [0, 1, 2]
Total Value: 12
>>>
```

Ln: 49 Col: 0